

SEARCHING

Any algorithm, which performs a search operation, accepts an argument we call as “ele” and tries to find an occurrence of “ele” in some structure. The commonly used structure to illustrate various methods is the array. It is possible that the search for a particular element in a structure is unsuccessful, i.e., there is no occurrence in the structure with the argument “ele” as its key. In such a case the algorithm should return a special message to indicate an unsuccessful search.

LINEAR SEARCH

The simplest of all forms of searching is the linear search. This search is applicable to a table organized either as an array or as a linked list. Let us assume that **A** is an array of **N** elements from **A[0]** through **A[N - 1]**. Let us also assume that **ele** is the search element. The search starts by sequentially comparing the elements of the array one after the other from the beginning to the end with the element to be searched. If the element is found its position is identified otherwise an appropriate message is displayed.

ALGORITHM:

Linear_Search(A , ele, N)

A is an array of N elements and ele is the element being searched in the array.

```
Step 1: For I = 0 TO N-1 Do
Step 2: If ( ele = A[I] ) Then
Step 3:         return( I )
           [EndIf]
           [End of For loop]
Step 4:         return (-1)
Step 5:         Exit
```

EFFICIENCY OF LINEAR SEARCH

The efficiency of linear search can be understood by examining the number of comparisons made by a linear search in searching for a given element. We assume that no insertions or deletions are performed, so that the searching is performed through an array of a constant size N. The number of comparisons depends on where the element appears in the array. If the element is in the first position of the array, only one comparison is to be performed, if the element is in the last position of the array then N comparisons are necessary to find the element. If it is equally likely for the argument to appear at any given array position, a successful search will take $(n + 1)/2$ comparisons, and an unsuccessful search will take n comparisons. On an average the number of comparisons is $O(n)$.

Program: To perform linear search:

```
#include <stdio.h>
#include <conio.h>

intLinear_search( int* , int , int );
void main()
{
    int    a[100], ele, pos, n, i;

    clrscr( );
```

```

printf(" \nEnter the size of the array :");
scanf("%d",&n);
printf(" \nEnter %d elements\n",n);
for(i = 0 ; i < n ; i ++ )
    scanf("%d", &a[i]);
printf(" \nEnter element to search :");
scanf("%d",&ele);

pos= Linear_search(a, ele,n);

if ( pos == -1)
    printf(" \nElement not present");
else
    printf(" \nElement found in position : %d", pos);
}

/*      Function to search an item using linear search technique      */

intLinear_search( int a [ ] , int e, int m)
{
    int    i;

    for(i = 0 ; i < m ; i ++ )
        if ( a[i]==e )
            return(i+1);
    return(-1);
}

```

Alternate Recursive function for Linear Search

```

int Linear_search(int a[], inte, int m)
{
    if (m <= 0)
        return (-1);
    else if (a[m-1] == e)
        return (m);
    else
        return LinearSearch(a, e, m-1);
}

```

BINARY SEARCH

The main disadvantage of linear search is the time taken to decide. The decision of an element not being present can be made only after comparing N element. This problem can be overcome when the elements are in sorted order with the technique called binary search. Binary search is considered the most efficient, method of searching a linear array without the use of auxiliary indices. The element to be compared is compared with the element of the middle element of the array. If the elements are the same, the search ends successfully; otherwise, the search should continue either to the left or to the right of the array in a similar manner. It may be noted that the binary search can best be defined recursively. However, the overhead associated with recursion may make it inappropriate for use in practical situations in which efficiency is a prime consideration. We will therefore look at a non-recursive version of the binary search algorithm

ALGORITHM:

Binary_Search(A , ele, N)

A is a sorted array of **N** elements and **ele** is the element being searched in the array. **LOW** and **HIGH** identify the positions of the first and last elements in a range and **MID** identifies the position of the middle element.

```

Step 1: LOW = 0
Step 2: HIGH = N - 1
Step 3: While (LOW <= HIGH) Do
Step 4:     MID = (LOW + HIGH) / 2
Step 5: If (ele == A[MID]) Then
Step 6:     return(MID)
Step 7: If (ele < A[MID]) Then
Step 8:     HIGH = MID - 1
        Else
Step 9:     LOW = MID + 1
        [End of Step While loop]
Step 10: return(-1)

```

EFFICIENCY OF BINARY SEARCH

Every comparison in the binary search reduces the number of possible elements remaining for comparisons by a factor of 2. Thus, the maximum number of element comparisons is approximately $\log_2 n$. We may say that the binary search algorithm is $O(\log n)$. If the size of the array is small then binary search is not more advantageous than a linear search. Unfortunately, the binary search algorithm can only be used if the array is stored. This is because it makes use of the fact that the indices of array elements are consecutive integers. For this reason the binary search is practically useless in situations where there are many insertions or deletions, so that an array structure is inappropriate.

Program: To perform binary search:

```

#include <stdio.h>
#include <conio.h>

int Binary_search( int *, int , int );

void main( )
{
    int    a[100], ele, pos, n, i;

    clrscr( );
    printf("\n Enter the array size ");
    scanf("%d",&n);
    printf("\n Enter %d elements in sorted order\n",n);
    for(i = 0 ; i < n ; i++)
        scanf("%d", &a[i]);
    printf(" Enter the element to search\n");
    scanf("%d",&ele);

    pos= Binary_search(a, ele,n);
    if ( pos == -1)
        printf(" Element not present \n");
    else
        printf(" Element found in position :%d\n", pos);
}

```

```

}

/*      Function to perform Binary search      */
intBinary_search( int a [ ] , int e, int m)
{
    int    low, high, mid;
    low = 0;
    high = m - 1;

    while (low <= high)
    {
        mid = (low + high) / 2;
        if (e == a[mid])
            return(mid+1);
        if (e < a[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
    return(-1);
}

```

Alternate Recursive function for Binary Search

```

intbinarySearch(inta[], intlow, inthigh, inte)
{
    int    mid;

    if (low < high)
    {
        mid = ( low + high) / 2;
        if (e<a[mid])
            returnbinarySearch(a, low, mid-1, e);
        else if (e>a[mid])
            returnbinarySearch(a, mid+1, high , e);
        else
            return(mid); // Found it.
    }
    return (-1);    // Failed to find element
}

```

INTRODUCTION TO SORTING

Every activity of life requires us to be organized for the effective execution of the job. The job of Data Processing requires us to have all the data in some order or the other. Ordering or sorting of data with some relationship is of fundamental importance, however certain factors should be considered before designing a sort algorithm. Algorithms are designed with the following objectives.

1. The movement of data should be as minimized as possible. If the size of the data item is large and if there is excessive movement of data, this would result in a large amount of processing time.
2. The movement of data items between the secondary storage and the main memory should be in large blocks. The large the data block the more efficient is the data block.

3. As much as possible the data should be retained in the main memory, so that it can be effectively used.

BUBBLE SORT

The algorithm of bubble sort functions as follows. The algorithm begins by comparing the element at the bottom of the array with the next element. If the first element is larger than the second element then they are swapped or interchanged. The process is then repeated for the next two elements. After $n-1$ comparisons the largest of all the items slowly ascends to the top of the array. The entire process till now forms one pass of comparisons. During the next pass the same steps are repeated from the beginning of the array, however this time the comparisons are only for $n-1$ elements. The second pass results in the second largest element ascending to its position. The process is repeated again and again until only two elements are left for comparison. The last iteration ensures that the first two elements of the array are placed in the correct order.

Suppose the list of numbers $A[0], A[1], A[2]$ and $A[3]$ is in memory.

33 44 22 11

The bubble sort algorithm works as follows:

Pass 1.

1. Compare $A[0]$ and $A[1]$ and arrange them in the desired order, so that $A[1] > A[0]$.
2. Then compare $A[1]$ and $A[2]$ and arrange them so that $A[2] > A[1]$.
3. Then compare $A[2]$ and $A[3]$ and arrange them so that $A[3] > A[2]$.

Continue until we compare $A[N-1]$ with $A[N-2]$ and arrange them so that $A[N-1] > A[N-2]$.

Comparison	Elements	Result	Array
$A[0]$ with $a[1]$	33 and 44	No interchange	33 44 22 11
$A[1]$ with $a[2]$	44 and 22	Interchange	33 22 44 11
$A[2]$ with $a[3]$	44 and 11	Interchange	33 22 11 44

Observe that Pass 1 involves $N-1$ comparisons. During Pass 1 the largest element is bubbled up to $n-1$ th position. When Pass 1 is completed, $A[N-1]$ will contain the largest element.

Pass 2.

Repeat Pass 1 with one less comparison i.e., now we stop after we compare and possible rearrange $A[N-3]$ and $A[N-2]$.

Comparison	Elements	Result	Array
$A[0]$ with $a[1]$	33 and 22	Interchange	22 33 11 44
$A[1]$ with $a[2]$	33 and 11	Interchange	22 11 33 11

When Pass 2 is completed, $A[N-2]$ will contain the second largest element.

Pass 3.

Repeat Pass 1 with two lesser comparison i.e., now we stop after we compare and possible rearrange $A[N-4]$ and $A[N-3]$.

Comparison	Elements	Result	Array
------------	----------	--------	-------

A[0] with a[1] 22 and 11 Interchange 11 22 33 44

i.e., we finally compare A[0] and A[1] and arrange them so that A[0] < A[1].

After n-1 passes, the list will be sorted in increasing order.

The following algorithm describes bubble sort for sorting the array of "N" elements in the ascending order:

BUBBLE SORT ALGORITHM

ALGORITHM: Bubble_Sort(A, N) Given an array A of N elements, this procedure sorts the elements in the ascending order using the method described above. The variables I and J are used to index the array elements.

```
Step 1: For I = 0 to N - 1 Do
Step 2: For J = 0 to N - I Do
Step 3: [ Compare adjacent elements ]
        If ( A [ J ] > A [ J + 1 ] ) Then
            [Exchange the values]
Step 4:     Temp    = A [ J ]
Step 5:     A [ J ]  = A [ J + 1 ]
Step 6:     A [ J + 1 ] = Temp
        [End If]
        [End of Step 2 For loop ]
        [End of Step 1 For loop ]
Step 7: Exit
```

ANALYSIS OF BUBBLE SORT

In analyzing the procedure of bubble sort, we shall count only the number of element comparisons. It is easy to see that the frequency count of all other operations is the same as that of the comparisons. In the algorithm we assume that all the elements are distinct.

The first pass of the algorithm results in N-1 comparisons and in the worst case may result in N-1 exchanges also. The second pass results in N-2 comparisons and in the worst case may result in N-2 exchanges. Continuing the analysis we observe that as the iterations or passes increases the comparisons and exchanges decreases. Finally the total number of comparisons will be equal to

$$\begin{aligned} &= (N-1) + (N-2) + (N-3) + \dots + 2 + 1 \\ &= (N) * (N-1) / 2 \\ &= O(N^2) \end{aligned}$$

Program: To perform bubble sort is as follows:

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int    a[100], n;
    int    i, j, temp;

    clrscr();
```

```

printf("\nEnter number of elements ");
scanf("%d",&n);
printf("\nEnter array elements\n");
for( i=0 ; i<n ; i++ )
    scanf("%d",&a[i]);

printf("\nThe unsorted list is\n");
for( i=0 ; i<n ; i++ )
    printf("\t%d",a[i]);
printf("\n");

for(i = 0; i < n ; i++)
for(j=0 ; j<n-i ; j++)
if(a[j] > a[j+1])
{
    temp = a[j];
    a[j] = a[j+1];
    a[j+1]=temp;
}

printf("\n The sorted list is \n");
for( i=0 ; i<n ; i++ )
    printf("\t%d",a[i]);
printf("\n");
getch();
}

```

SELECTION SORT

This is a sorting algorithm, which is very simple to understand and implement. The algorithm achieves its name from the fact that with each iteration the appropriate value for a key position is selected from the list of remaining elements and put in the required position of the array. However the algorithm is not efficient for large arrays. The method of selection sort relies heavily on a comparison mechanism to achieve its goals.

ALGORITHM:

Suppose an array A with n elements A[1], A[2], ..., A[N] is in memory. The selection sort algorithm for sorting A works as follows.

- Step1: First find the smallest element in list and put it in the first position.
- Step2: Find the second smallest element in the list and put it in the second position.
- Step3: Repeat the process for all the positions one after the other from the beginning to the end.

ALGORITHM:

Selection_Sort(A, N) Given an array A of N elements, this procedure sorts the elements in the ascending order using the method described above. The variables I and J are used to index the array elements.

- Step 1 : For I = 1 TO N -1 Do
- Step 2 : [Assume ith elements as smallest]
- small = A [I]

```

Step 3 :      POS = I
Step 4 : [Find the smallest element in the array and its position ]
           For J = I + 1 TO N Do
Step 5 : If ( A [ J ] < small ) Then
Step 6 :      small = A [ J ]
Step 7 :      POS = J
           [EndIf]
           [End of Step 4 For loop]
Step 8 : [Exchange ith element with smallest element]
           A[ POS ] = A [ I ]
Step 9 : A [ I ] = small
           [End of Step 1 For loop]
Step 10: Exit

```

ANALYSIS OF SELECTION SORT

In analyzing the procedure of selection sort, we shall count only the number of element comparisons. It is easy to see that the number of exchanges is the same as that of the number of elements. In the algorithm we assume that all the elements are distinct.

The first pass of the algorithm results in $n-1$ comparisons. The second pass results in $n-2$ comparisons. Continuing the analysis we observe that as the iterations or passes increases the comparisons decreases. Finally the total number of comparisons will be equal to

$$\begin{aligned}
 &= (n-1) + (n-2) + (n-3) + \dots + 2 + 1 \\
 &= (n)(n-1) / 2 \\
 &= O(n^2)
 \end{aligned}$$

We observe that in spite of the superiority of the selection sort algorithm over the bubble sort algorithm, there is no significant gain in run time. The efficiency of the algorithm still remains as $O(n^2)$ for "N" data items.

Program: To perform selection sort is as follows:

```

#include <stdio.h>
#include <conio.h>
void main()
{
    int    a[100], n;
    int    i, j, small, pos;

    clrscr();
    printf("\nEnter number of elements ");
    scanf("%d",&n);
    printf("\nEnter array elements\n");
    for( i=0 ; i<n ; i++ )
        scanf("%d",&a[i]);

    printf("\nThe unsorted list is\n");
    for( i=0 ; i<n ; i++ )
        printf("\t%d",a[i]);
    printf("\n");

    for(i = 0; i < n-1 ; i++)
    {

```

```

        small = a[i];
        pos = i;
        for(j=i+1 ;j<n ;j++)
            if(a[j] < small)
            {
                small=a[j];
                pos = j;
            }
        a[pos] = a[i];
        a[i] = small;
    }

    printf("\n The sorted list is \n");
    for( i=0 ; i<n ; i++)
        printf("\t%d",a[i]);
    printf("\n");
    getch();
}

```

INSERTION SORT

This is one of the natural ways using which information can be sorted. It follows the same procedure that is used by card game players. The main idea of the algorithm is to build a complete solution by inserting a new element from the unsorted portion of a list into the appropriate position in the partially ordered portion of the list. The procedure is extended one element at a time for the entire list. The method employed is similar to the one used in selection sort where we select the smallest element from the unordered part of the list and place it at the end of the sorted part.

The algorithm of insertion sort functions as follows. Initially to start the whole array is in a completely unordered state. To begin with the second element is considered as the element to be inserted from the unordered part of the list. The first element is considered to be in the ordered part. The second element is then inserted either in the first or the second position as appropriate. This process extends the ordered part to two elements and the remaining elements are considered to be unordered. Inserting the third element, then the fourth element and so on slowly extends the ordered part. The outline for the insertion sort algorithm is as follows

ALGORITHM:

```

For ele = 2 to N do
begin
    Step 1 : Choose the next element for insertion
              i.e., x = a [ I ]
    Step 2 : Insert the chosen element "x" into the ordered part
              of the array
end

```

ALGORITHM:

Insertion_Sort(A, N) Given an array **A** of **N** elements, this procedure sorts the elements in the ascending order using the method described above. The variables **I** and **J** are used to index the array elements.

```

Step 1:      For I = 2 to N Do
Step 2:      J = I

```

```

Step 3:      While( J >= 2)
Step 4:      If ( A [ J ] < A [ J - 1 ] ) then
Step 5:          temp    =  A [ J ]
Step 6:          A [ J ]  =  A [ J - 1 ]
Step 7:          A [ J - 1 ] =  temp
              [EndIf]
Step 8:      J = J - 1
              [End of While loop]
              [End of Step 1 For loop]
Step 9:      Exit

```

ANALYSIS OF INSERTION SORT

In analyzing the procedure of insertion sort, we will have to count the number of element comparisons and the number of array elements that need to be shifted or moved. It is easy to see that the number of exchanges is the same as that we have seen in bubble sort. In the algorithm we assume that all the elements are distinct.

The first pass of the algorithm results in one comparison and in the worst case may result in one exchange also. The second pass results in two comparisons and in the worst case may result in two exchanges. Continuing the analysis we observe that as the iterations or passes increases the comparisons and exchanges decreases. Finally the total number of comparisons will be equal to

$$\begin{aligned}
 & 1 + 2 + 3 + \dots + (n-3) + (n-2) + (n-1) \\
 = & (n) * (n-1) / 2 \\
 = & O(n^2)
 \end{aligned}$$

Program: To perform insertion sort.

```

#include <stdio.h>
#include <conio.h>
void main()
{
    int    a[100], n;
    int    i, j, temp;

    clrscr();
    printf("\nEnter number of elements ");
    scanf("%d",&n);
    printf("\nEnter array elements\n");
    for( i=0 ; i<n ; i++ )
        scanf("%d",&a[i]);
    printf("\nThe unsorted list is\n");
    for( i=0 ; i<n ; i++ )
        printf("\t%d",a[i]);
    printf("\n");
    for(i =1; i < n ; i++ )
    {
        j = i;
        while ( j >= 1)
        {
            if(a [j] < a [j-1] )
            {
                temp = a[j];
                a[j] = a[j-1];
                a[j-1]= temp;
            }
        }
    }
}

```

```

        }
        j = j - 1 ;
    }
}
printf("\nThe sorted list is\n");
for( i=0 ; i<n ; i++ )
    printf("\t%d",a[i]);
printf("\n");
getch();
}

```

QUICK SORT

This is one of the best techniques for a large set of data. This technique also works on the method of partitioning. The process of divide and conquer is again recursively applied. In the merge sort method of sorting, the array was partitioned exactly at the midpoint, however in this method of sorting the partition is created at a position such that the elements to the left of the partition is less than the elements to the right of the partition. The entire process functions as follows. First we choose an element from a specific position in the array to be sorted (for example let the element "ele" be chosen as the first element of the array "a" so that ele = a[0]). Suppose that the elements of the array a are partitioned so that ele is placed into the position "p" and the following conditions are satisfied.

1. Each of the elements in the positions 0 through p-1 is less than or equal to ele.
2. Each of the elements in the positions j+1 through n-1 is greater than or equal to ele.

The purpose of quick sort is to move the data item in the correct direction just enough for it to reach its final place. Thus the amount of swapping is greatly reduced and the required item moves a greater distance in a shorter duration of time. Let us now trace the entire sequence for the following set of data.

26 55 46 38 12 91 81 35

The process of performing quick sort involves the following steps.

1. Remove the first data item, 26, mark its position and scan the array from left to right comparing all the data items with the value 26. When an element which is greater than 26 is found make a note of its position.
2. Scan the array from right to left comparing all the data items with the value 26. When an element which is smaller than 26 is found make a note of its position.
3. Interchange the positions of the smaller and the greater number, which has been found with the above steps.
4. Perform the steps 1,2 and 3 until the scanning from left to right and from right to left meet or cross each other. This is the position where the first data item is to be placed.
5. Partition the array at this position and we observe that all the elements to the left of the partition are less than the elements to the right of the partition. The steps described above are then repeated for each partition separately.

Let us now illustrate the steps described above on the example

The steps are initially repeated on the initial array and at the end of one set of comparisons the first element finds its position and the array assumes the following structure.

12 26 55 46 38 91 81 35

We observe that all the elements to the left of 26 is smaller and all elements to the right is greater. Since 26 is in its final position the original problem is decomposed into two sub-arrays as follows.

(12) and (55 46 38 91 81 35)

It can be seen that nothing can be done with the first sub-array. To sort the second sub-array the process is to be repeated. Initially the array is viewed as follows.

12 26 (55 46 38 91 81 35)

When the process is repeated the partitions created are as follows

12	26	(46	38	35)	55	(91	81)
12	26	(38	35)	46	55	(91	81)
12	26	(35)	38	46	55	(91	81)
12	26	35	38	46	55	(91	81)
12	26	35	38	46	55	(81)	91
12	26	35	38	46	55	81	91

Finally the entire array is sorted.

Observing the steps described above we note that the entire process can be performed recursively. The algorithm to perform quick sort may be described in the following steps.

ALGORITHM:

Quick_Sort(A, low, high) Given an array A of **high - low + 1** elements, this procedure sorts the elements in the ascending order using the method described above. The variables **low** and **high** are used to identify the positions of the first and last elements in each partition.

Step 1:	If (low < high) then
Step 2:	Call Partition(A , low , high, pos)
Step 3:	Call Quick_sort(A , low, pos -1)
Step 4:	Call Quick_sort(A , pos + 1, high)
Step 5:	[EndIf]
Step 6:	Exit

The main aim of the partition procedure is to find a proper position for a specific element; the manner in which the partition is performed is irrelevant to the sorting method. Here we strictly follow the technique the described above to create the partition. Two pointers named “up” and “down” will be used to initialize the lower bound and the upper bound of the partition. At any instant in the partition each element to the right of the pointer “up” is greater than or equal to the element at the pointer position and each element to the left of the pointer “down” is less than or equal to the element at the pointer position. The two pointers are slowly moved towards each other and the point where they cross each other is the correct position of the specified element.

ALGORITHM:

Partition (a, low, high, pos) Given an array A of **high - low + 1** elements, this procedure partitions the elements into two lists using the method described above. The variables **low**, **high** and **pos** are used to identify the positions of the elements in each partition. The first partition is from the position low to the position **pos - 1** and the next partition is from the position **pos + 1** to the position high.

Step 1: $x = a[\text{low}]$

```

Step 2: up = high
Step 3: down = low
Step 4: While ( down < up ) Do
Step 5: While ( a [ down] <= x) and ( down < high )Do
Step 6:         down = down + 1
           [End of Step 5 While loop]
Step 7:         While ( a [ up] > x) Do
Step 8:         up = up - 1
           [End of Step 7 While loop]
Step 9:         If ( down < up ) Then
Step 10:         temp = a [ down ]
Step 11:         a [ down ] = a[ up]
Step 12:         a [ up] = temp
           [EndIf]
           [End of Step 4 While loop]
Step 13: a [ low ] = a [ up ]
Step 14: a [ up ] = x
Step 15: pos = up                                /* position to be returned */
Step 16:         Exit

```

ANALYSIS OF QUICK SORT

In analyzing quick sort we shall count the number of element comparisons $C(n)$. It is observed that the frequency count of all other operations are of the same order $C(n)$. The following assumptions are also made during the analysis.

1. The elements of the array are distinct.
2. The partition element is chosen using a random selection process.
3. The proper position for the pivot element always turns out to be the middle of the sub array.
4. The array size is assumed to be a power of 2 , say $n = 2^m$ or $m = \log_2 n$.

In that case there will be approximately be n comparisons in the first pass, after which the array is split into two sub arrays each approximately of the size $n/2$. For each of the sub arrays there are approximately about $n/2$ comparisons and a total of four sub arrays each of size $n/4$ are formed. Each of the sub arrays then requires $n/4$ comparisons and yielding $n/8$ sub arrays. After repeating the process “ m ” times then there will be “ n ” sub arrays each of size 1. Thus the total number of comparisons is approximately equal to

$$\begin{aligned}
 C(n) &= n + 2 * (n / 2) + 4 * (n / 4) + + n * (n / n) \\
 &= n + n + n + + n (m \text{ times}) \\
 &= O(n * m) \\
 &= O(n \log n)
 \end{aligned}$$

Program: To perform quick sort.

```

#include <stdio.h>
#include <conio.h>

void quicksort(int* , int, int);
void partition(int*, int, int, int* );
void main()
{
    int a[20], n, i;

    clrscr();
    printf(" Enter the number of elements \n");

```

```

scanf("%d",&n);
printf(" Enter %d elements \n",n);
for(i=0;i<n;i++)
    scanf("%d",&a[i]);
printf(" The UnSorted elements are\n");
for(i=0;i<n;i++)
    printf("\t%d",a[i]);
    printf("\n");
quicksort( a, 0, n-1 );
printf(" The Sorted elements are\n");
for(i=0;i<n;i++)
    printf("\t%d",a[i]);
    printf("\n");
}

/*      Recursive Quick sort function      */
void quicksort(int a[], int low, int high)
{
    int    pos;
    if ( low < high )
    {
        partition( a , low , high, &pos);
        quicksort( a , low, pos -1 );
        quicksort( a , pos + 1, high);
    }
}

/*      Function to create partitions      */
void partition (int a[], int low, int high, int *p)
{
    int    x, up, down,temp;

    x = a[low] ;
    up = high;
    down = low ;
    while ( down < up )
    {
        while ( (a [ down] <= x) && ( down < high ))
            down = down + 1 ;
        while ( a [ up] > x)
            up = up - 1;
        if ( down < up )
        {
            temp = a [ down ];
            a [ down ] = a[ up];
            a [ up] = temp;
        }
    }
    a [ low ] = a [ up ];
    a [ up ] = x;
    *p = up;
}

```

MERGE SORT

This sorting method follows the technique of divide-and-conquer. The technique works on a principle where a given set of inputs are split into distinct subset and the required method is applied on each

subset separately i.e., the sub problems are first solved and then the solutions are combined into a solution of the whole. Most of the times the sub problems generated will be of the same type as the original problem. In such situations re-application of the divide-and-conquer technique may be necessary on each sub problem. This is normally achieved through a recursive procedure. Thus smaller and smaller sub problems are generated until it is possible for us to solve each sub problem without splitting.

The technique of merge sort works as follows. Given a sequence of “n” elements the idea is to split them into two sets. Each set is individually sorted and the resulting sequence is then combined to produce a single sorted sequence of “n” elements. If each subset is of the same type as the original set then the subset is further recursively divided into smaller subsets until each subset is small enough to be solved independently without splitting. The sorted subsets are then combined to obtain a single solution to the entire problem.

The technique described above can be performed in the following steps

1. Divide the sequence of elements into two equal parts.
2. Recursively sort the elements on the left of the division.
3. Recursively sort the elements on the right of the division.
4. Merge the sorted left and right parts into a single sorted array.

The algorithm of merge sort may have the following steps

ALGORITHM:

Merge_Sort(A, low, high) Given an array **A** of **high - low + 1** elements, this procedure sorts the elements in the ascending order using the method described above. The variables **low** and **high** are used to identify the positions of the first and last elements in each partition.

```

Step 1:      If ( low < high ) Then
Step 2:          mid = ( low + high ) / 2
Step 3:      Call Merge_sort( A, low , mid)
Step 4:      Call Merge_sort( A, mid+1, high)
Step 5:      Call Merge( A , low, mid, high)
              [EndIf]
Step 6:      Exit

```

The statement $low < high$ with the if statement allows the procedure to be applied until it is not possible to split the problem. The first two recursive procedure calls continues the process of splitting until each sub problem can be solved independently. The third procedure call combines the solved sub problems into a single solution. The process of dividing a problem into sub problems can be clearly understood by the example shown below.

(66	22	10	30	41	51	82	77	2)
(66	22	10	30	41)	(51	82	77	2)
(66	22	10)	(30	41)	(51	82)	(77	2)
(66	22)	(10)	(30)	(41)	(51)	(82)	(77)	(2)
(66)	(22)	(10)	(30)	(41)	(51)	(82)	(77)	(2)

The process of combining or merging two sorted arrays can be explained as follows. Merging of two or more sets of data is frequently performed in computing. It is simpler than sorting because it is possible to take advantage of the partial order of the data. Consider the following two arrays

A: 17 21 45 52

B: 9 12 18 19 43 58

The merging operation results in a new array C and will finally have the elements in the following sequence. The origin of each element is also indicated.

	B	B	A	B	B	A	B	A	A	B
C	9	12	17	18	19	21	43	45	52	58

The following facts can be observed from the original and resultant arrays of the example given above. The resultant array is normally longer and contains elements corresponding to the sum of the elements of the individual arrays. It can also be seen that to construct the resultant array it is necessary to examine all the elements of both the individual arrays. Thus it is necessary for to generate an appropriate mechanism for selecting elements the individual arrays. Finally the order placing elements in the resultant array will also be necessary.

ALGORITHM:

Merge(a, low, mid, high) Given an array **A** of **high - low + 1** elements, this procedure sorts the elements In the ascending order using the method described above. The variables **low**, **mid** and **high** are used to identify the positions of the elements In each partition. The first partition Is from the position low to the position mid and the next partition Is from the position mid + 1 to the position high.

```

Step 1:      I = low
Step 2:      J = mid + 1
Step 3:      K = low
Step 4:      While ( I <= mid ) and ( J <= high ) Do
Step 5: I      If ( a[I] < a[J] )
Step 6:          c[K] = a[I]
Step 7:          K = K + 1
Step 8:          I = I + 1
              Else
Step 9 :      c[K] = a[J]
Step 10:      K = K + 1
Step 11:      J = J + 1
              [EndIf]
              [End of Step 4 While loop]
Step 12: While ( I <= mid ) Do
Step 13:      c[K] = a[I]
Step 14:      K = K + 1
Step 15:      I = I + 1
              [End of Step 12 While loop]
Step 16: While ( J <= high )
Step 17:      c[K] = a[J]
Step 18:      K = K + 1
Step 19:      J = J + 1
              [End of Step 16 While loop]
Step 20: For I = low to high
Step 21:      a[I] = c[I]
              [End of Step 20 For loop]

```

Step 22: Exit

ANALYSIS OF MERGE SORT

The following assumptions are also made during the analysis.

1. The elements of the array are distinct.
2. The partition element is chosen using a random selection process.
3. The proper position for the pivot element always turns out to be the middle of the sub array.
4. The array size is assumed to be a power of 2, say $n = 2^m$ or $m = \log_2 n$.

In that case there will be approximately n comparisons in the first pass, after which the array is split into two sub arrays each approximately of the size $n/2$. For each of the sub arrays there are approximately about $n/2$ comparisons and a total of four sub arrays each of size $n/4$ are formed. After repeating the process " m " times then there will be " n " sub arrays each of size 1. Thus the total number of comparisons is approximately equal to

$$\begin{aligned} C(n) &= n + 2 * (n / 2) + 4 * (n / 4) + + n * (n / n) \\ &= n + n + n + + n \text{ (m times)} \\ &= O(n * m) \\ &= O(n \log n) \end{aligned}$$

Program: To perform merge sort.

```
#include <stdio.h>
#include <conio.h>
void mergesort( int*, int, int);
void merge( int*, int, int, int );
void main()
{
    int a[20], n, i;

    clrscr();
    printf("\nEnter the number of elements ");
    scanf("%d",&n);
    printf(" \nEnter %d elements \n",n);
    for(i=0;i<n;i++)
        scanf("%d",&a[i]);
    printf(" \nTheUnSorted elements are\n");
    for(i = 0; i<n;i++)
        printf("\t%d",a[i]);
    printf("\n");
    mergesort(a, 0, n-1);
    printf(" \nThe Sorted elements are\n");
    for(i = 0; i < n; i++)
        printf("\t%d",a[i]);
    printf("\n");
}
/* Merge_sort function */
void mergesort(int a[ ], int low, int high)
{
    int mid;
    if ( low < high )
    {
        mid = (low + high) / 2;
        mergesort( a , low, mid );
```

```

        mergesort( a , mid + 1, high);
        merge(a,low,mid,high);
    }
}
/*      Merge      function      */
void merge (int a[], int low, int mid, int high)
{
    int    i, j, k,c[20];
    i = low ;
    k = low;
    j = mid + 1 ;
    while ( ( i <=mid) && ( j <= high))
    {
        if ( a[i] < a[j] )
        {
            c[k] = a[i];
            k++;
            i++;
        }
        else
        {
            c[k] = a[j];
            k++;
            j++;
        }
    }
    while ( i <= mid)
    {
        c[k] = a[i];
        k++;
        i++;
    }
    while ( j <= high)
    {
        c[k] = a[j];
        k++;
        j++;
    }
    for ( i = low; i <= k-1; i++)
        a[i]= c[i];
}

```

Heap Sort Algorithm

Heap sort is one of the sorting algorithms used to arrange a list of elements in order. Heapsort algorithm uses one of the tree concepts called **Heap Tree**. In this sorting algorithm, we use **Max Heap** to arrange list of elements in Descending order and **Min Heap** to arrange list elements in Ascending order.

Step by Step Process

The Heap sort algorithm to arrange a list of elements in ascending order is performed using following steps...

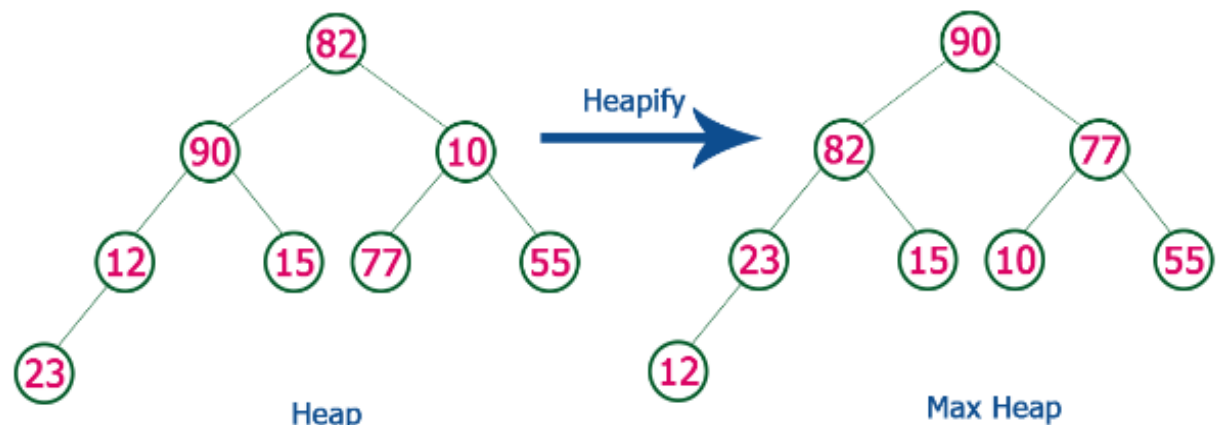
- **Step 1** - Construct a **Binary Tree** with given list of Elements.
- **Step 2** - Transform the Binary Tree into **Min Heap**.
- **Step 3** - Delete the root element from Min Heap using **Heapify** method.
- **Step 4** - Put the deleted element into the Sorted list.
- **Step 5** - Repeat the same until Min Heap becomes empty.
- **Step 6** - Display the sorted list.

Example

Consider the following list of unsorted numbers which are to be sort using Heap Sort

82, 90, 10, 12, 15, 77, 55, 23

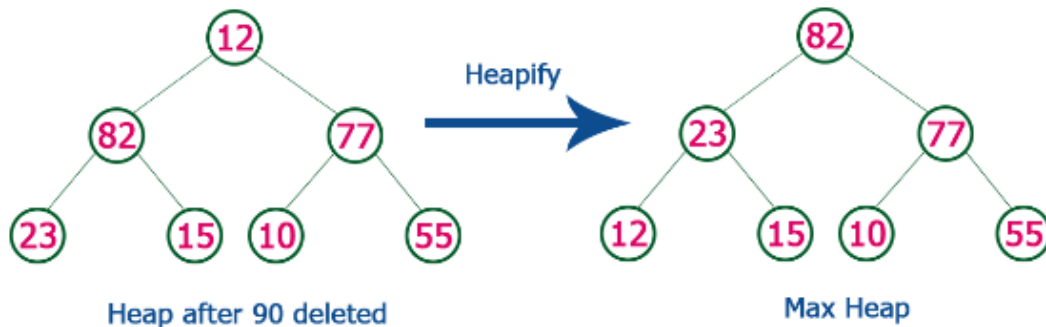
Step 1 - Construct a Heap with given list of unsorted numbers and convert to Max Heap



list of numbers after heap converted to Max Heap

90, 82, 77, 23, 15, 10, 55, 12

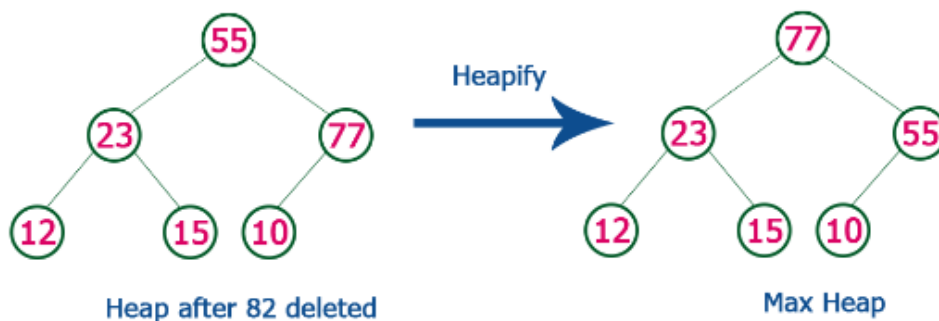
Step 2 - Delete root (**90**) from the Max Heap. To delete root node it needs to be swapped with last node (**12**). After delete tree needs to be heapify to make it Max Heap.



list of numbers after swapping 90 with 12.

12, 82, 77, 23, 15, 10, 55, 90

Step 3 - Delete root (**82**) from the Max Heap. To delete root node it needs to be swapped with last node (**55**). After delete tree needs to be heapify to make it Max Heap.



list of numbers after swapping 82 with 55.

12, 55, 77, 23, 15, 10, 82, 90

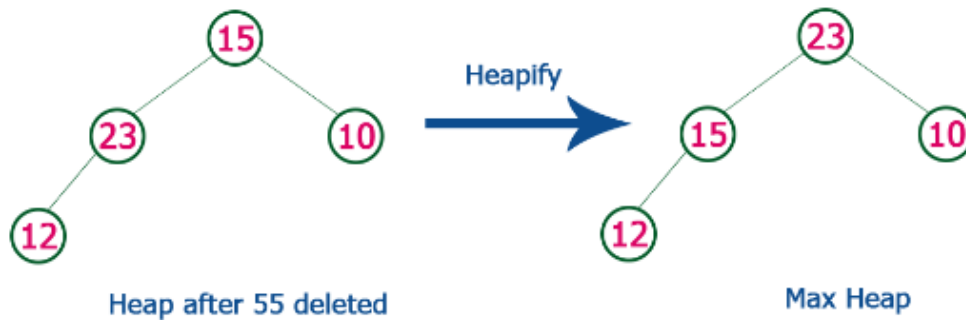
Step 4 - Delete root (**77**) from the Max Heap. To delete root node it needs to be swapped with last node (**10**). After delete tree needs to be heapify to make it Max Heap.



list of numbers after swapping 77 with 10.

12, 55, 10, 23, 15, 77, 82, 90

Step 5 - Delete root (**55**) from the Max Heap. To delete root node it needs to be swapped with last node (**15**). After delete tree needs to be heapify to make it Max Heap.



list of numbers after swapping 55 with 15.

12, 15, 10, 23, 55, 77, 82, 90

Step 6 - Delete root (**23**) from the Max Heap. To delete root node it needs to be swapped with last node (**12**). After delete tree needs to be heapify to make it Max Heap.

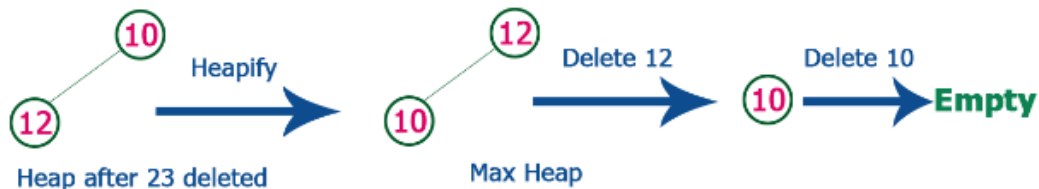


list of numbers after swapping 23 with 12.

12, 15, 10, 23, 55, 77, 82, 90

.....

Step 7 - Delete root (**15**) from the Max Heap. To delete root node it needs to be swapped with last node (**10**). After delete tree needs to be heapify to make it Max Heap.



list of numbers after Deleting 15, 12 & 10 from the Max Heap.

10, 12, 15, 23, 55, 77, 82, 90

Whenever Max Heap becomes Empty, the list get sorted in Ascending order

Complexity of the Heap Sort Algorithm

To sort an unsorted list with 'n' number of elements, following are the complexities...

Worst Case : $O(n \log n)$

Best Case : $O(n \log n)$

Average Case : $O(n \log n)$

Radix Sort Algorithm

Radix sort is one of the sorting algorithms used to sort a list of integer numbers in order. In radix sort algorithm, a list of integer numbers will be sorted based on the digits of individual numbers. Sorting is performed from **least significant digit to the most significant digit**. Radix sort algorithm requires the number of passes which are equal to the number of digits present in the largest number among the list of numbers. For example, if the largest number is a 3 digit number then that list is sorted with 3 passes.

Step by Step Process

The Radix sort algorithm is performed using the following steps...

- **Step 1** - Define 10 queues each representing a bucket for each digit from 0 to 9.
- **Step 2** - Consider the least significant digit of each number in the list which is to be sorted.
- **Step 3** - Insert each number into their respective queue based on the least significant digit.
- **Step 4** - Group all the numbers from queue 0 to queue 9 in the order they have inserted into their respective queues.
- **Step 5** - Repeat from step 3 based on the next least significant digit.

- **Step 6** - Repeat from step 2 until all the numbers are grouped based on the most significant digit.

Example

Consider the following list of unsorted integer numbers

82, 901, 100, 12, 150, 77, 55 & 23

Step 1 - Define 10 queues each represents a bucket for digits from 0 to 9.



Step 2 - Insert all the numbers of the list into respective queue based on the Least significant digit (once placed digit) of every number.

82, 901, 100, 12, 150, 77, 55 & 23



Group all the numbers from queue-0 to queue-9 in the order they have inserted & consider the list for next step as input list.

100, 150, 901, 82, 12, 23, 55 & 77

Step 3 - Insert all the numbers of the list into respective queue based on the next Least significant digit (Tens placed digit) of every number.

100, 150, 901, 82, 12, 23, 55 & 77



Group all the numbers from queue-0 to queue-9 in the order they have inserted & consider the list for next step as input list.

100, 901, 12, 23, 150, 55, 77 & 82

Step 4 - Insert all the numbers of the list into respective queue based on the next Least significant digit (Hundred placed digit) of every number.

100, 901, 12, 23, 150, 55, 77 & 82



Group all the numbers from queue-0 to queue-9 in the order they have inserted & consider the list for next step as input list.

12, 23, 55, 77, 82, 100, 150, 901

List got sorted in the increasing order.

Complexity of the Radix Sort Algorithm

To sort an unsorted list with 'n' number of elements, Radix sort algorithm needs the following complexities...

Worst Case : $O(n)$

Best Case : $O(n)$

Average Case : $O(n)$